

Slowcedes: DeepRacer Challenge catch-up

Everything we built, learned and changed from the first baseline to Model 05 · status as of 2026-09-30 evening

Where we stand. Team **Slowcedes** has **100% completion** on the secret evaluation track with a **score of 18.008** (best lap 14.710 s), down from 34.7 → 22.5 → 18.0 over three uploads. The leader (Shallow Learner) is at 5.615. Our best model is **Model 04 final** (uploaded). **Model 05** (racing line + smooth steering) is training now on the local RTX 3090.

The five things to know

- **We train locally** with DeepRacer-for-Cloud (DRfC) on the 3090 in WSL2 (allowed by the organizers' email), with heavy automation because the simulator is fragile on Windows.
- **The car only sees one camera image.** Our reward teaches it by imitating a track-agnostic **expert driver** that we compute from the track's waypoints (pure pursuit + braking-aware speed profile + grip limit). No hard-coded positions, so it follows the rules and generalizes.
- **Curriculum:** each model continues from the previous one's best checkpoint and makes the task harder (slow laps → speed → both directions → faster → racing line).
- **We judge models on unseen tracks**, not on Vegas: 4-track evaluation, time including off-track penalties, and a **generalization efficiency** metric (theoretical best ÷ actual lap).
- **The portal counts off-tracks as time, not as failure:** completion stayed 100% even when our model went off; fewer off-tracks is what cut our score from 22.5 to 18.0.

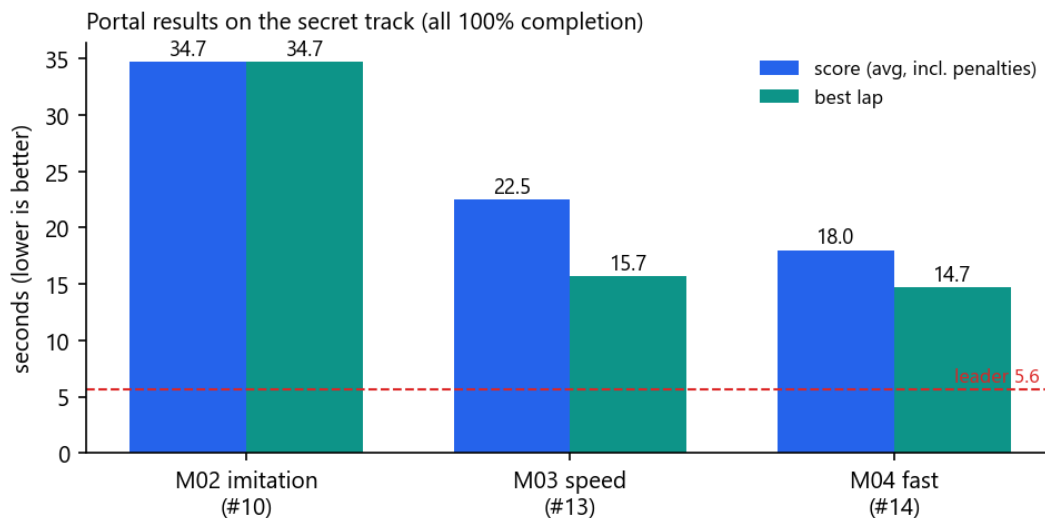


Fig. 1: Our three portal submissions vs. the leader.

1. The challenge in one page

We train an AWS DeepRacer car (1/18 scale) with reinforcement learning. Models are uploaded to the CU Denver portal (deepracer.ashishkb.info) and evaluated on a **secret track** (3 trials). The leaderboard ranks **completion rate first, then the lower score** (time-based). Selected teams race the same model on a **physical car on October 8**; the practice race (Vegas) closes October 7.

Rule (organizers' email)	How we comply
Single camera only	FRONT_FACING_CAMERA
PPO only	clipped PPO, continuous action space
Train on the public Vegas track (AWS Summit Raceway)	All training is on Vegas_track (both directions). Other tracks are only used to <i>evaluate</i> .
Generalize, avoid hard-coding waypoints	The reward computes everything from runtime <i>waypoints</i> of whatever track it is on
Local infrastructure allowed (e.g. DRfC)	Yes, DRfC on our own RTX 3090
Keep notes, change one or two things at a time	experiments/LOG.md + a README per model

Open question for the organizers (still to send): is training on simulator tracks other than Vegas allowed, and is evaluating on them locally OK? Until answered we train on Vegas only.

2. How we train: local setup

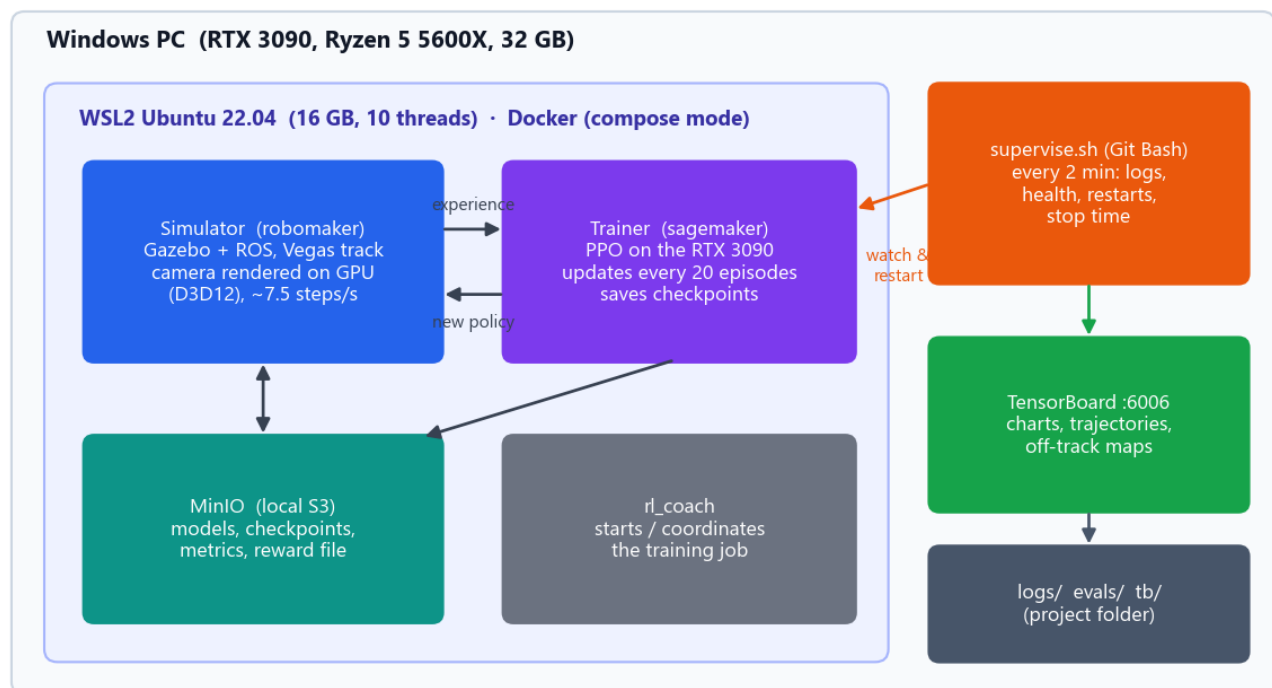


Fig. 2: Local training system. Simulator and trainer run in Docker inside WSL2; a supervisor script on Windows keeps everything alive and logs to the project folder.

Getting DRfC stable on a Windows desktop took most of the first day. Every problem below happened to us and is now handled automatically (details in docs/DRFC_SETUP.md):

Problem we hit	Symptom	Fix now in place
MinIO image removed from Docker Hub (Sep 2026)	storage would not start	Chainguard MinIO image, re-tagged
Simulator rendered on CPU	2.8 steps/s	GPU rendering via WSL D3D12 (GALLIUM_DRIVER=d3d12) → ~9.8 steps/s
Docker swarm manager timeouts	simulators silently shut down	switched to compose mode
2 simulators + Windows apps	CPU load 12/12, physics timeouts	1 simulator
WSL given 24 GB + 8 GB swap	Windows at 0.8 GB free, WSL froze	.wslconfig: 16 GB, 10 threads, 4 GB swap, autoMemoryReclaim
WSL idles its VM when no Windows process attached	training killed after terminal closed	supervisor holds a keep-alive connection
Simulator memory leak with GPU rendering	~1 GB/min until the PC chokes	restart <i>only</i> the simulator; the trainer keeps its state
Simulator restarted mid-iteration	trainer waits forever (silent stall)	restart only at iteration boundaries + stuck-trainer detection (8 min → full resume)
DRfC auto-increment bug	resumed runs pointed at nonexistent parents	own resume script names runs -2, -3...
Several metrics files with 2 workers	episode counts halved	exporter merges them
<i>is_reversed</i> means 'clockwise', not 'wrong way'	clockwise episodes got 0 reward	removed from all rewards
Interrupted evaluation left another track in run.env	trained 1 min on the wrong track	start script always forces Vegas

3. The RL problem

Only the camera image reaches the network; the reward (training only) sees waypoints & position.

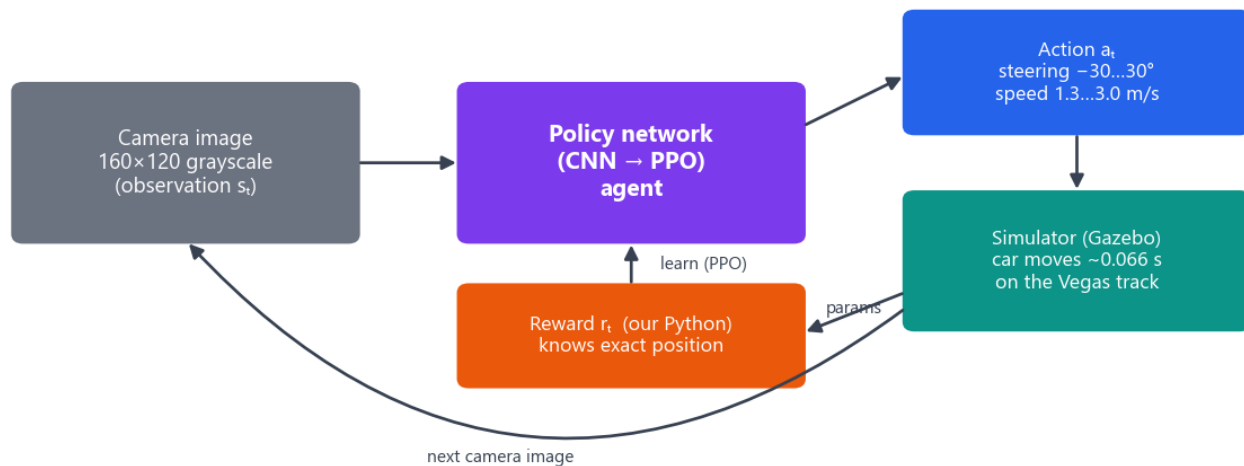


Fig. 3: The reinforcement-learning loop (~15 decisions per simulated second).

RL concept	In our setup
Agent	Policy network (CNN), clipped PPO actor-critic
Observation	Only the front camera: 160×120 grayscale, 1 frame (stack_size 1)
Action	Continuous: steering $-30\dots+30^\circ$, speed 1.3...3.0 m/s (Model 04/05)
Reward	Our Python function, sees exact position, waypoints, heading, speed... (training only)
Episode	Random start point; ends on lap complete or off track. Direction alternates every episode



Car's camera



What the network actually gets (160×120 gray)



Simulator broadcast view

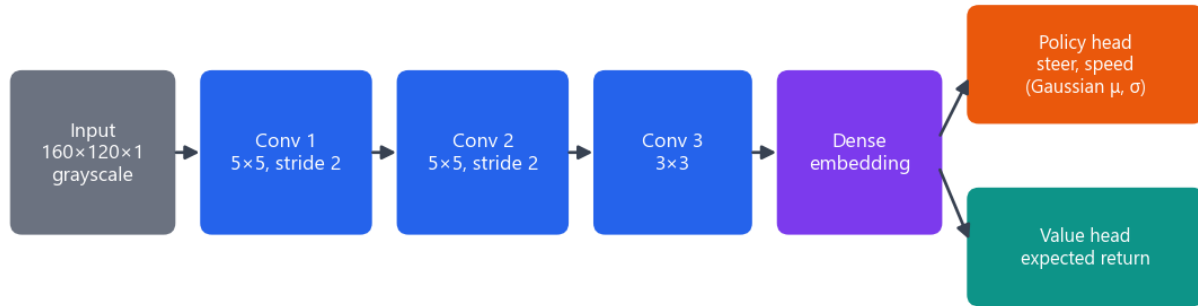


Fig. 4: Network architecture (DeepRacer's shallow CNN).

Hyperparameters & throughput

Hyperparameter	Value	Meaning
learning rate	0.0003	step size of each update
discount γ	0.99	looks ~100 steps (≈ 6.6 s, a third of a lap) ahead
episodes between training	20	one iteration = 20 episodes, then an update
epochs / batch size	5 / 64	≈ 230 gradient steps per update
entropy β	0.01	exploration bonus
term_cond_max_episodes	1000	Note: a single run may stop learning after 1000 episodes; raise it for overnight runs

Measured: 15 decisions per simulated second; the simulation runs at about **half real time** (≈ 7.5 steps/s); ~ 2.5 – 3 episodes/min; one iteration ≈ 8 – 10 min; **≈ 150 – 180 episodes and ~ 6 – 7 updates per hour**. The GPU is mostly idle: the simulator (CPU) is the bottleneck. All hyperparameters are still DRfC defaults.

4. Reward design: imitate an expert driver

Instead of rewarding 'stay near the centre' (the starter reward), every step the reward computes what a simple expert would do **on this track** and rewards the car for doing something similar. The network never sees the expert; it must learn the behaviour from pixels, which is what transfers to unseen and physical tracks.

- **Steering:** pure pursuit, aiming at a point 0.6–0.83 m ahead (further at higher speed) on the path.
- **Speed:** a braking-aware speed profile. Every curve ahead allows $\sqrt{(\text{grip} \times \text{radius})}$; the car must be able to brake (3 m/s^2) in time for it. Plus a grip cap from the steering angle ($\leq 4 \text{ m/s}^2$ lateral).
- **Model 05 additions:** the path is a **racing line** (K1999-style curvature averaging, 0.30 m from the edges), and a **smooth-steering bonus** for small steering changes.
- **Lap bonus:** $+100 \times (\text{average lap speed} / 2 \text{ m/s})$, so faster laps pay more.

Model 05 reward: imitate a track-agnostic expert (racing line + speed planner)

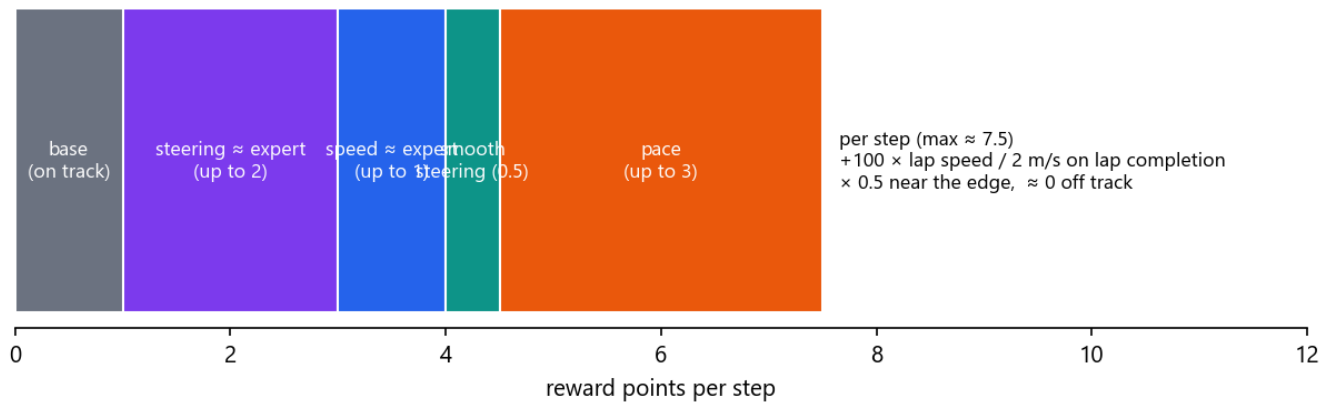


Fig. 5: Reward per step (Model 05).

Vegas: centre line vs. computed racing line (tightest radius $0.58 \rightarrow \sim 0.8$ m)

--- centre line (M02–M04 expert)
— racing line (M05 expert)

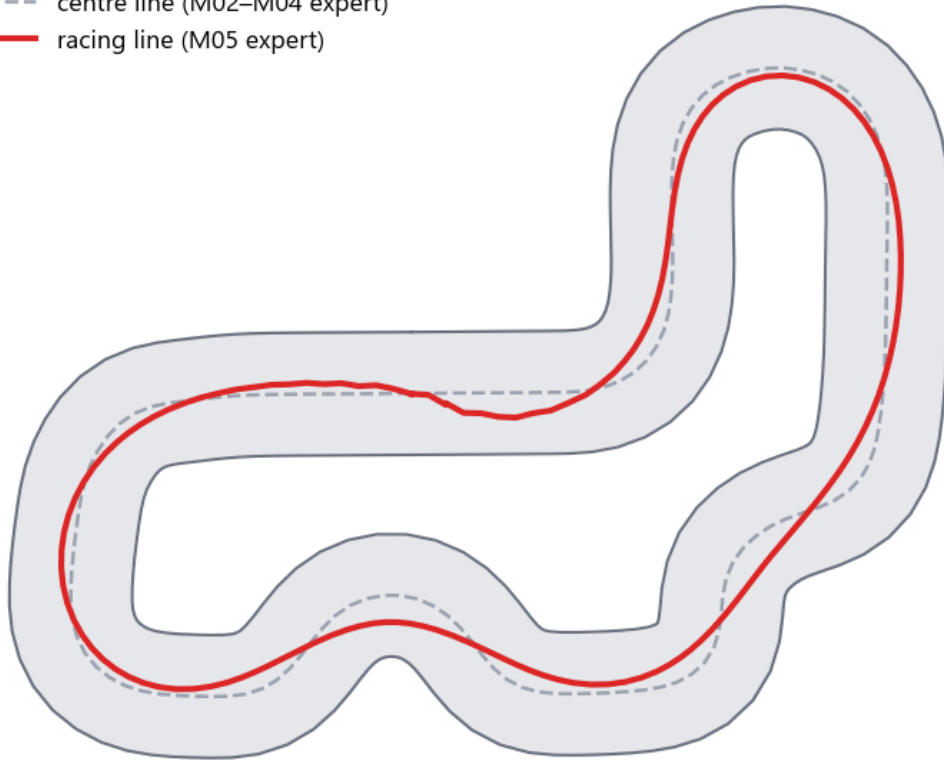


Fig. 6: Centre line vs. computed racing line on Vegas. Same algorithm runs on any track.

Every reward is first tested offline (tools/test_reward.py): a simple car model drives each track using only the expert's commands. Model 05's expert laps all 10 test tracks and is $\sim 10\%$ faster than the centre-line expert (Vegas 8.2 vs 9.1 s). Two earlier versions failed this test and were fixed before training.

5. Model history (curriculum)

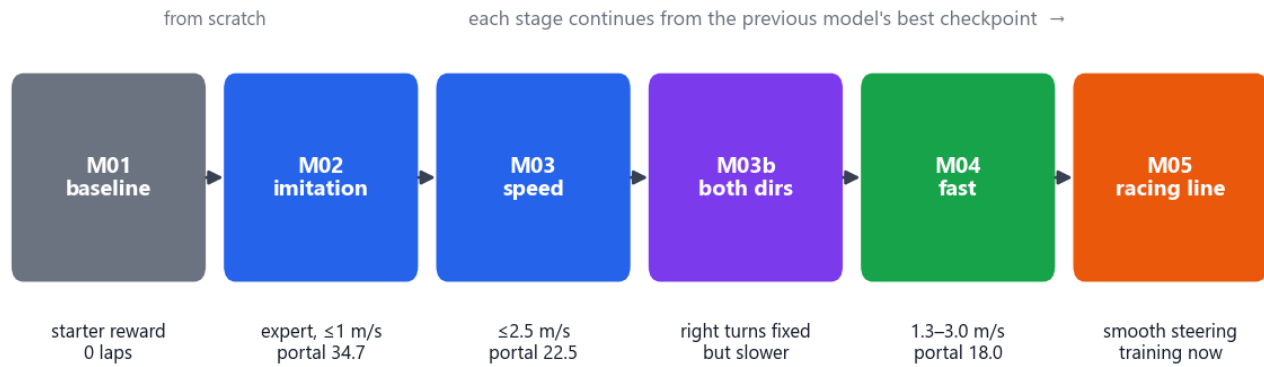
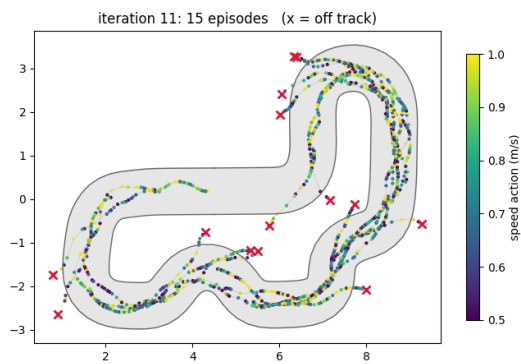


Fig. 7: Each stage continues from the previous model's best checkpoint.

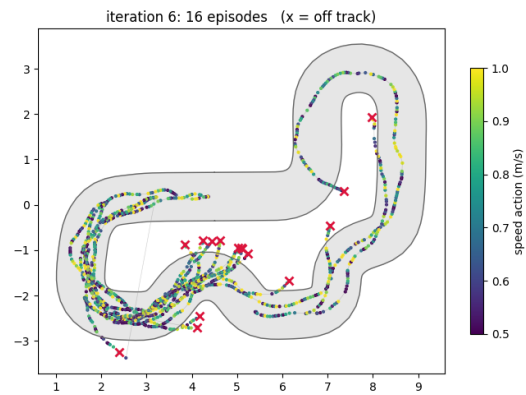
Model	What changed	Started from	Outcome
M01 baseline	Organizers' starter reward (stay near centre), 0.5–1.0 m/s	scratch	0 laps in 225 episodes; heavy zig-zag (23°/step)
M02 imitation	Expert reward (pure pursuit + slow before curves)	scratch	First laps; Vegas 3/3 clean, 27 s; portal 34.7 . Off-tracks only in right turns
M03 speed	1.0–2.5 m/s, braking speed profile + grip limit	M02 ckpt 11	Vegas 14.7 s; portal 22.5 ; more off-tracks on unseen right-handers
M03b both dirs	Alternate driving direction each episode	M03 ckpt 31	Right turns fixed (11 vs 30 off-tracks) but slower (1.46 → 1.28 m/s); not uploaded
M04 fast	1.3–3.0 m/s + lap bonus scaled by lap speed	M03b final	Mean 17.0 s on 4 tracks, half the off-tracks; portal 18.0
M05 racing line	Racing line + smooth-steering bonus	M04 final	Training now (best lap 13.1 s so far)

Trajectories during training

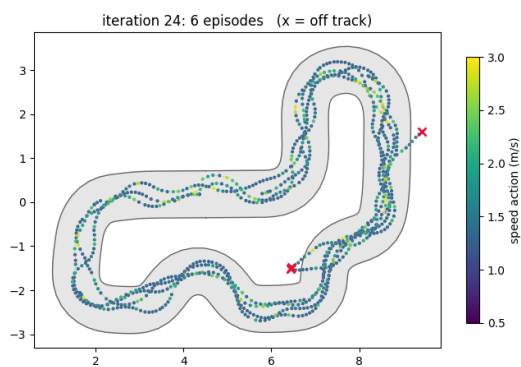
Each plot shows the paths of one training iteration on Vegas, coloured by commanded speed, with × where the car left the track. Note the speed scales differ (0.5–1.0 m/s for M01/M02, up to 3.0 m/s for M04/M05).



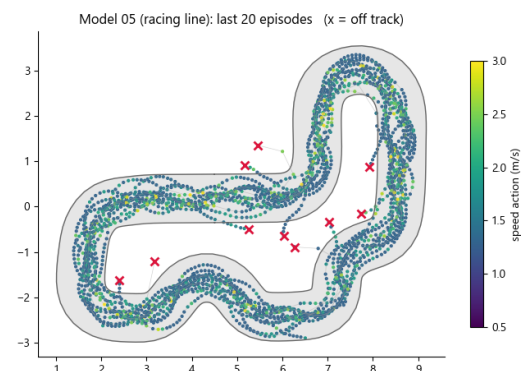
M01 baseline: scattered paths, off-tracks everywhere, slow



M02 imitation (early): follows the road, crashes cluster in the S-section



M04 fast: consistent laps at 1.3–3.0 m/s, 1 off-track in 6 episodes



M05 racing line (now): fast laps, both directions, few off-tracks

6. How we evaluate (and why Vegas alone lies)

Every candidate checkpoint is evaluated locally under practice-race rules (3 trials, individual lap, resets allowed, 1 s off-track penalty) on **4 tracks**: Vegas plus three it never trained on (Summit Speedway, re:Invent 2018, re:Invent 2024 clockwise). We save hourly snapshots and evaluate several, because the last checkpoint is not always the best (overnight, 03b got slower the longer it trained).

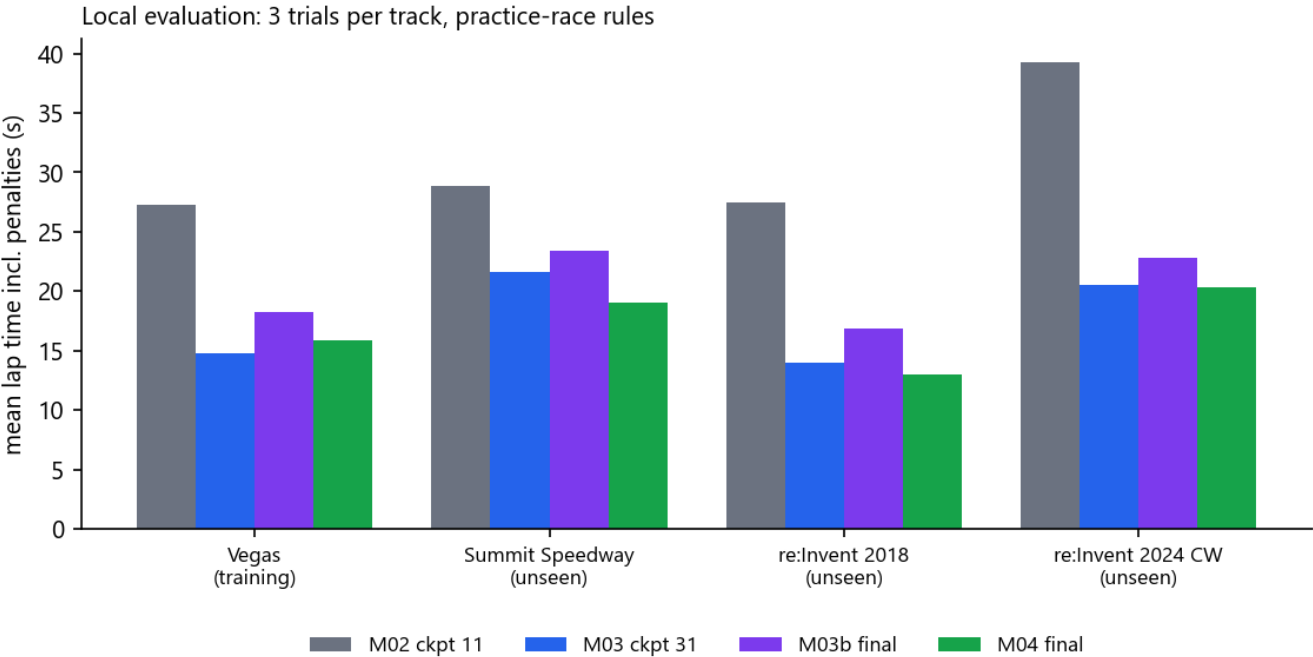


Fig. 8: Mean time per track including penalties (lower is better).

Model	Vegas	Summit	re:Inv 2018	re:Inv 2024 CW	Mean	Off-tracks /12	Portal
M02 ckpt 11	27.2	28.8	27.5	39.2	30.7	27	34.716
M03 ckpt 31	14.7	21.6	14.0	20.5	17.7	30	22.508
M03b final	18.3	23.4	16.9	22.8	20.3	11	—
M04 final	15.8	19.0	13.0	20.3	17.0	15	18.008

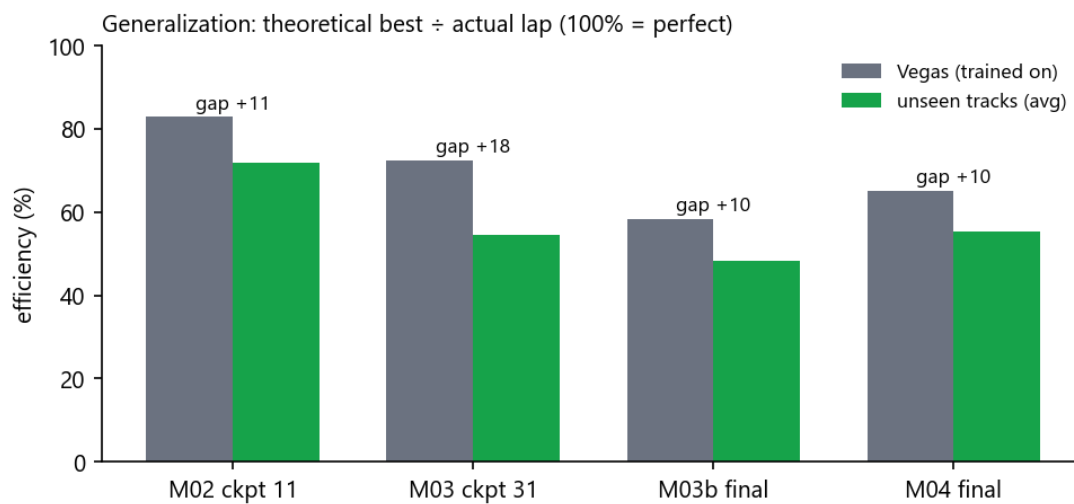


Fig. 9: Generalization efficiency. A model that memorized Vegas scores much higher there than on unseen tracks (large gap). Both-direction training cut the gap from 18 to 10 points.

7. What limits us: lap-time simulation & measured physics

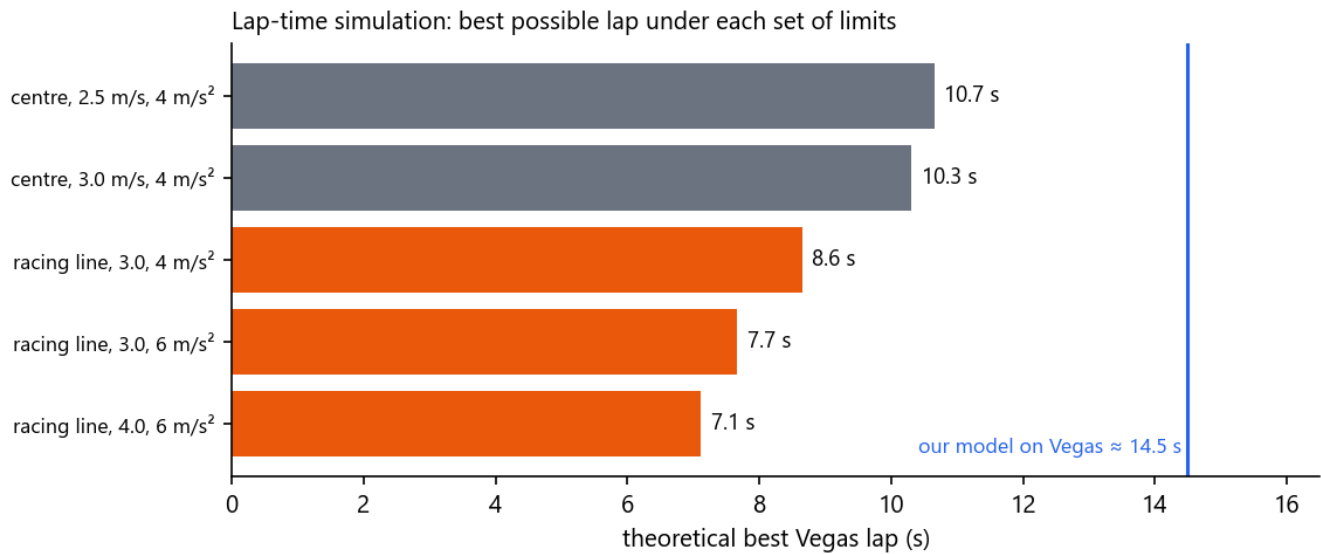


Fig. 10: Theoretical best Vegas lap under each set of limits (quasi-steady-state lap simulation, tools/lap_time_sim.py).

- Our model drives at **~60% of what its own limits allow** (≈ 14.5 s vs 8.65 s theoretical): execution, not limits, is the main gap.
- A racing line is worth ~ 1.6 s per Vegas lap; more top speed barely helps (grip-limited, short straights).
- **The leader's 5.54 s** is below even the 4 m/s / 6 m/s² bound (7.1 s) for a Vegas-length track. Either the secret track is much shorter, they use a top speed above 4 m/s (DRfC allows it), or the timing differs. Worth asking.

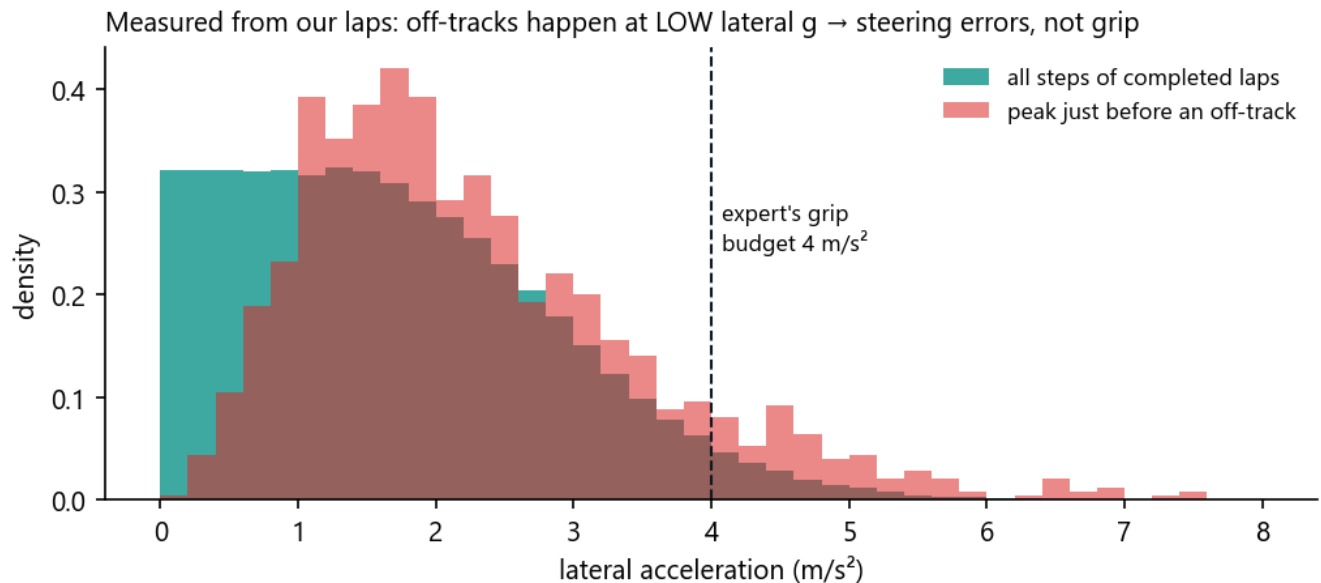


Fig. 11: Lateral acceleration measured from 994 completed laps vs. the moment before 1,246 off-tracks.

- The simulator holds **at least 5–6 m/s²** lateral (p99 4.8, p99.9 6.0): the expert's 4 m/s² is conservative.
- **Off-tracks happen at low lateral g** (median 2 m/s²): the car isn't sliding off, it steers wrong or late. That is a skill problem.
- Top-speed test: commanded 6 m/s on a 5.7 m straight, the car accelerated at ~ 4.5 m/s² and reached 3.89 m/s with no visible cap; inconclusive (track too short), and our tracks never let it pass ~ 3.4 m/s anyway.

8. Current state & next steps

Model 05 (racing line), live at 18:50	Value
Episodes / laps	154 / 83
Last 20 episodes	13 laps, 77% mean progress
Best / mean lap (Vegas training)	13.1 s (record) / 14.8 s
Clockwise / counterclockwise lap	14.2 s / 15.2 s
Steering change	15.3°/step (M01: 23.3°)

Plan

- Finish Model 05 (hourly snapshots), evaluate on the 4 tracks with efficiency; upload if it beats M04's 17.0 s mean.
- Raise the expert's grip budget to $\sim 5 \text{ m/s}^2$ (measured), and make the line use more of the track.
- Hyperparameter experiments (one at a time): lower learning rate for fine-tuning, more episodes per iteration, lower entropy.
- **Control experiment (good for the second PC):** train Model 05's reward *from scratch* to check whether the curriculum helps or carries old habits.
- Physical race (Oct 8): pick the most reliable model (fewest off-tracks, both directions); real cameras punish zig-zag steering.

How you can help

- Set up DRfC on your PC with **docs/DRFC_SETUP.md** (copy our .wslconfig, compose mode, GPU-rendering fix, MinIO fix).
- Run the from-scratch control or a hyperparameter experiment in parallel; use **tools/supervise.sh** to babysit it.
- Send the organizers the open question (other tracks for training/evaluation; how 'best lap' is measured).

9. Cheat sheet: files & tools

Path	What it is
README_CHALLENGE.md	Challenge overview, rules, practice race details
experiments/LOG.md	One row per model + portal history
experiments/modelNN-*/	reward_function.py, model_metadata.json, README with results
experiments/ANALYSIS_limits.md	Lap-time simulation, measured grip, generalization efficiency
experiments/OVERNIGHT_2026-09-30.md	Overnight log of Model 03b
docs/DRFC_SETUP.md, docs/RL_FORMULATION.md	Setup guide (all fixes) and RL explanation
tools/supervise.sh HH:MM	Babysits a run: logs, TensorBoard, memory-leak restarts, stall detection, stop time
tools/wsl/*_sh	WSL helpers: start_run, autoresume, simrestart, evalrun, eval_candidates, snapshot, ...
tools/test_reward.py	Offline test of a reward's expert on 10 tracks
tools/run_summary.py, direction_summary.py, checkin.sh	Progress summaries (overall and per direction)
tools/tb_export.py → tensorboard --logdir tb	TensorBoard: scalars, PPO stats, trajectories, off-track maps
tools/lap_time_sim.py, grip_from_logs.py, efficiency.py	Analysis tools from section 6–7
submissions/*_tar.gz	Validated portal bundles (made with cedic_package_model.py)